

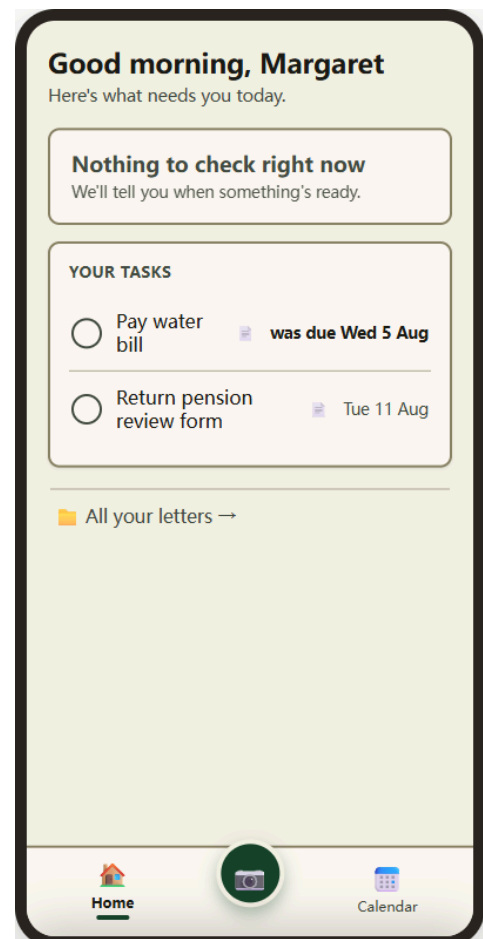
DayKeeper

A walk through the product, and the machinery behind each screen

This document follows one letter from the moment it is photographed to the moment the task it became is ticked off. Every screen you see is the working prototype; under each one is what the system does, which tables it writes, and why it was designed that way rather than another way.

It describes the shape of the system, not the state of the work. The board says who is doing what this week; this says what the thing is.

Written by Junchun Zhang, technical lead
For Dr Golnoush Abaei (supervisor), and the team
Screens the clickable prototype, docs/prototype/user/
Version 1.0



The pile

Margaret Wilson is 78. She lives alone in a brick house in the northern suburbs of Melbourne, her cataracts are getting worse, and every bill she has still arrives on paper. Her daughter is in Sydney and helps by phone. She has a smartphone, and she uses it for calls and for photographs.

Once a week she clears the pile that has built up by the front door. She photographs it the way anyone would: whatever is on top, then whatever is under that. What she posts looks like this.

Nine photographs, taken in one go

This is what actually arrives. Margaret is 78, her cataracts are getting worse, and every bill she has still comes on paper. She photographs the week's post the way anyone would: whatever is on top, then whatever is under it. Nothing in this pile is sorted, one of them is not a letter at all, and two of them cannot be read.

1

Example Energy
CALDERFIELD CITY COUNCIL
10 September 2026
Balance: \$82.42

2

Valuation and Rates Notice
CALDERFIELD CITY COUNCIL
10 September 2026
Margaret Wilson

3

Your electricity charges
CALDERFIELD CITY COUNCIL
10 September 2026
Balance: \$82.42

An electricity bill
page 1 of 2

A council rates notice
page 1 of 2

The electricity bill again
page 2, two photos later

4

Her grandson
not a letter

5

Thornbury Health
28 May 2026
Margaret Wilson

6

Valuation and Rates Notice - further information
CALDERFIELD CITY COUNCIL
10 September 2026

The rates notice again
page 2

7

A registration renewal
too blurred to read

8

A doctor's note
handwriting, unreadable

9

Two letters, one photograph
lying side by side on the table

Read the pile again, in order. Photograph 3 belongs with photograph 1, and there is a different letter in between. Photograph 9 holds two letters at once. Photograph 4 never becomes anything. Photograph 7 is a real letter that cannot be read, and photograph 8 is one we have to apologise for. **There is no rule that sorts this.** That is why the reading, not our code, decides where the letters divide.

What she wants back
One list of what to do, and the day it has to be done. Nothing else.

everything in
this
document
happens in
here

Read it again in order and the problem states itself. The electricity bill is photographs one and three, with a different letter in between. Photograph nine holds two letters at once, because they were lying side by side on the table. One is a picture of her grandson. One is a real letter too blurred to read, and one is handwriting that nothing will ever read.

There is no rule that sorts this. Every rule anybody proposes encodes an expectation, and the defining property of this input is that there is no expectation to encode. Somebody picking photographs out of an album destroys any convention before it exists.

That is the sentence the rest of this document answers. Nearly every decision here traces back to that pile: why the reading and not our code decides where the letters divide, why no screen asks her to confirm any of it, why the whole product has only two verbs, and why the calendar is built so that a value nobody checked has no path to reach it.

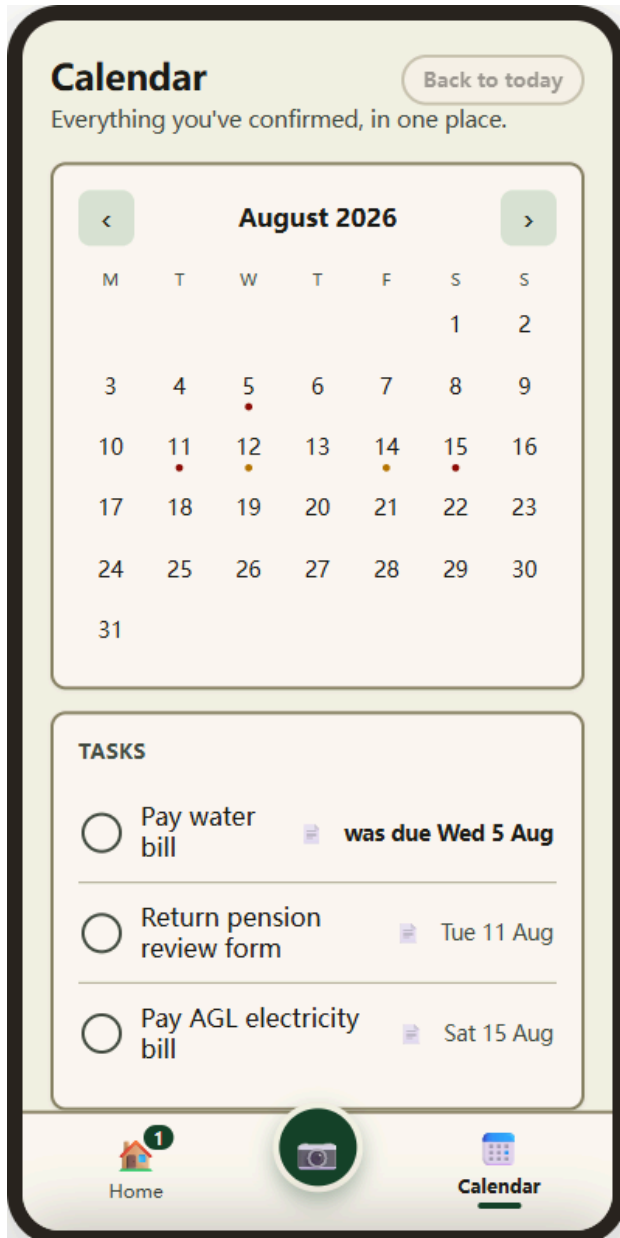
One thing to know before the screens. The prototype has a fixed fictional today: **Monday 10 August 2026**. Every date in every screenshot is relative to it, which is why the water bill due on the fifth is already late

DayKeeper · P000473SE · Group 1

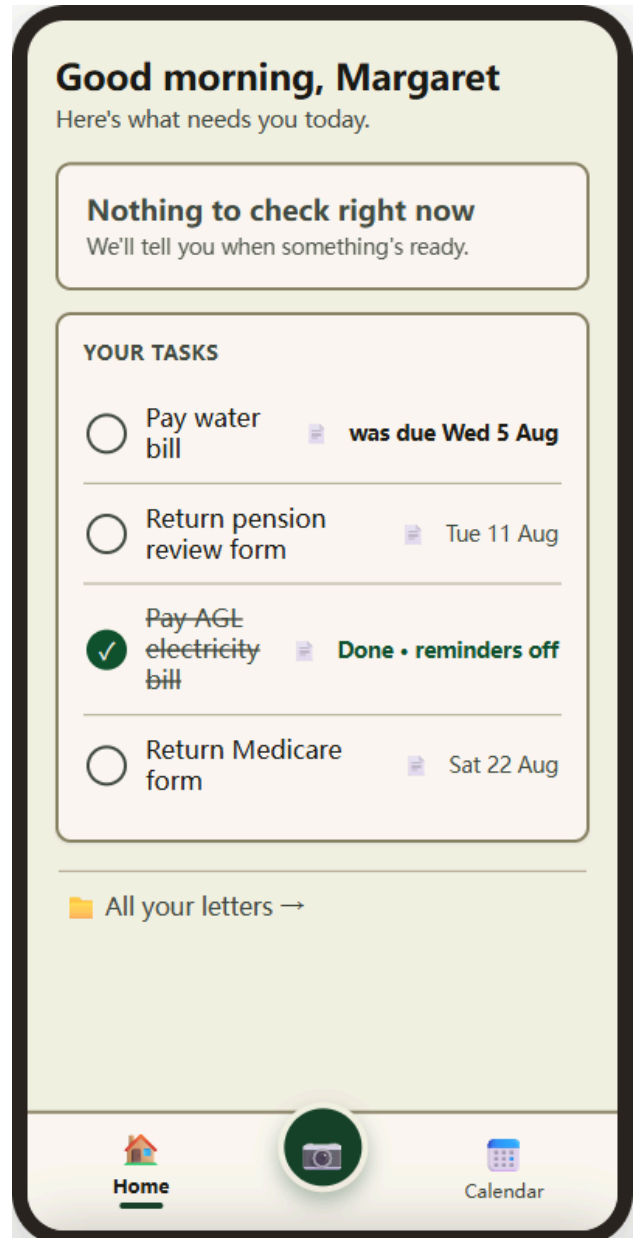
2

and the pension form due on the eleventh is not. The prototype wears this date in its toolbar so nobody has to guess.

What she gets back



The calendar. Red is a due date, gold is a morning she will be reminded.



The list. Overdue in words at the top, upcoming below, done and struck through but still on the list.

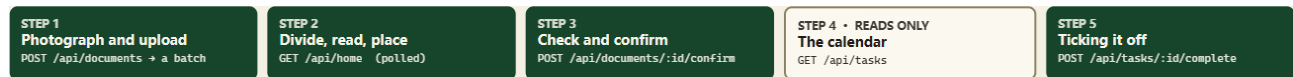
One list of what to do and the day it has to be done, and a calendar drawn from that same list. From where she is standing, that is the whole product.

The way she operates it is **two verbs: photograph, and tick**. She never types a date, never sorts pages, never files anything, never names a document, and never opens a settings screen to make any of it work. Everything else in this document exists to keep that true.

Nothing happens until you say so, and never from a date you haven't checked.

That promise is on the review screen, and it is the sentence this document keeps coming back to. The interesting part is that it is not kept by being careful. It is kept by there being no path.

Who she is matters to the design more than usual, because the people this product is for are the ones least able to absorb a mistake it makes. Eyes over seventy, a memory that is not reliable, and a lifetime of assuming that when a computer says no it is their own fault. So body text clears 7:1 contrast rather than the usual 4.5:1, nothing is blue, colour is never the only signal, and an error message never opens by describing something she did.

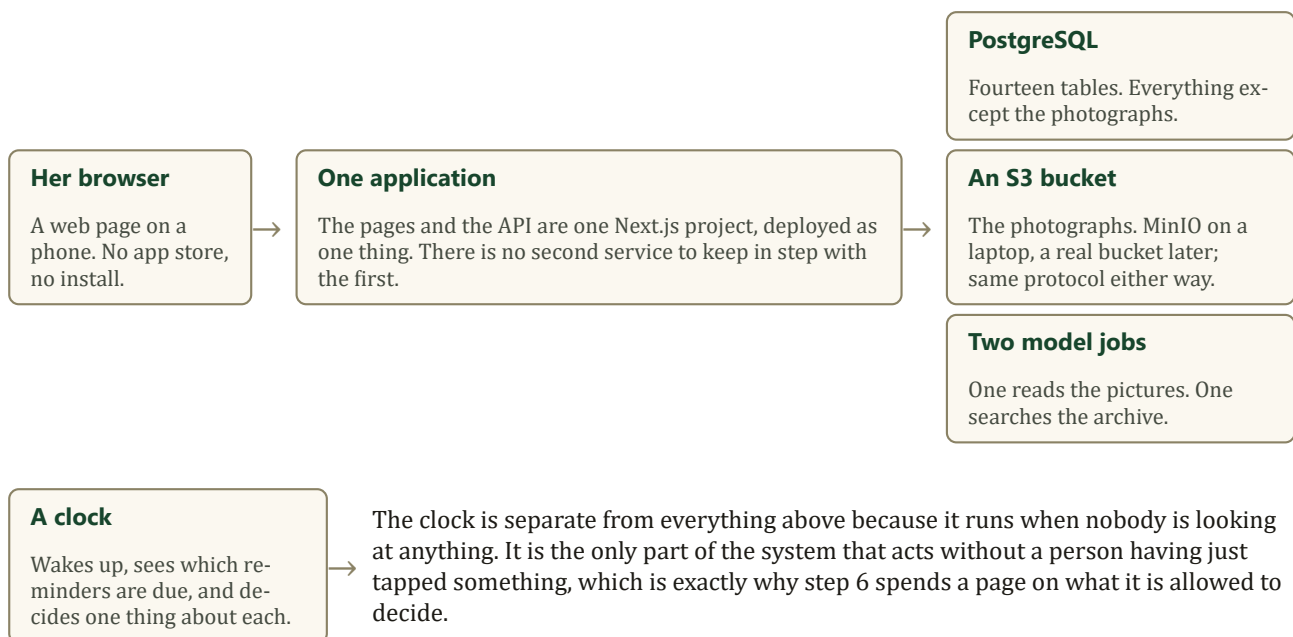


The five steps a letter goes through, which are also the middle of this document and the strip along the top of the database map on page 22.

Five parts follow. This one. Then one letter's whole life, from the photograph to the tick, which is five steps and most of the pages. Then what happens when it does not go well, which is where a product like this is actually judged. Then the fourteen tables, where a model is called, and the decisions behind all of it. Finally what is still open, and where to start reading if you are about to change something.

The pieces

Seven things do all the work. Everything later in this document is one of them doing its job.



One application rather than two. The project description names React and Next.js in the same breath as Python and FastAPI, which reads as an invitation to build a front end talking to a Python API. The scaffold this project stands on was already a single Next.js application, and the decision was to keep it that way. A second service is a permanent tax: two dependency sets, two deployments, a network boundary to authenticate across, and a second set of types kept identical to the first by hand. The one honest argument for Python is that the AI ecosystem lives there, and it does not apply here, because calling a vision model is an HTTP request in any language.

Before anything: whose nine in the morning

Two tables come before the story starts, because everything else points at them. `users` is a person: an email, a hashed password, a display name, a role. `sessions` is the fact that somebody is signed in right now, kept as a row so that signing out, or losing a phone, can end it immediately.

One column on `users` deserves a sentence of its own, because it is the kind of thing that is invisible until it is wrong. `timezone` holds an IANA zone name, defaulted to `Australia/Melbourne`. The product promises to remind her at nine in the morning, and nine in the morning is meaningless until you know whose morning. The same column decides when a task stops being due today and starts being overdue: the answer is midnight where she lives, not midnight where the server lives.

users	Who she is. Email, password hash, display name, role, and the zone her mornings happen in.
sessions	A signed-in browser, as a row. The cookie holds a random token; the table holds its hash, so a database leak does not hand over live sessions.

STEP 1 She photographs a pile

Writes: `upload_batches` `upload_pages`



The capture screen. Four photographs so far, and the camera stays open.

She points the camera at whatever came in the post. Not one letter: the pile. The screen says so in as many words, because the instinct is to photograph one thing and be careful about it, and being careful is the work this product is supposed to be doing for her.

When she presses the button, two tables are written and the request returns. `upload_batches` is one row for the pile she just posted. `upload_pages` is one row per photograph, holding where the file went, what format it is, and how big.

The photographs themselves are never in the database. The rows hold a path; the bytes are objects in a bucket. A database that holds images is a database whose backups are enormous and whose every query drags megabytes around for no reason.

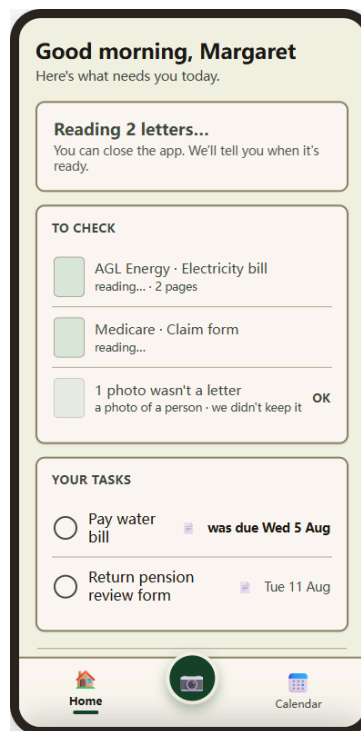
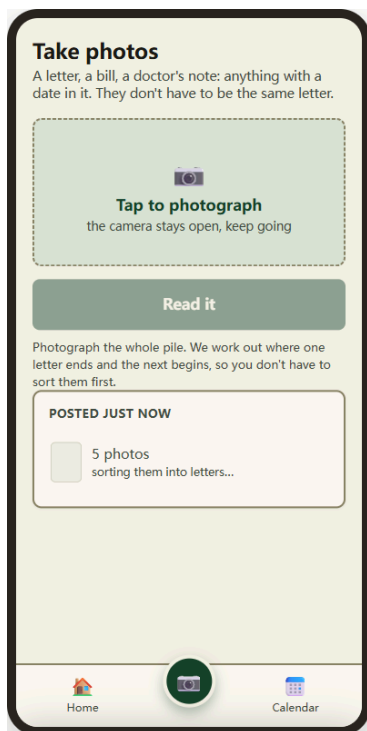
The upload answers as soon as the files are stored, not when the reading is done. Reading takes seconds, and a person holding a phone should not be watching a spinner for any of them.

The number on each photograph means less than it looks like it means. `upload_pages.position` says where a photograph fell in the pile, and that is all it says. It does not say which letter the photograph belongs to, and it does not say what order that letter's pages read in. Somebody picking photographs out of their phone album destroys any such convention before it exists. The number survives for two jobs: it is the name the reading points at ("photograph 3"), and it is the only thing left to show her if the reading fails completely.

There are two limits, ten photographs and ten megabytes each, and they live in one file that both the interface and the server import. That is the difference between a capture screen that stops her at the tenth photograph and one that lets her take fifteen and then rejects the lot.

STEP 2 The pile becomes letters

Writes: `grouping_runs` `documents` `document_pages`



Left: the pile has been posted and nothing has looked at it yet, so the row says the only honest thing it knows, which is how many photographs it holds. Right: seconds later the same row has become one row per letter, plus one grey line for the photograph that was not a letter at all.

This is the step the rest of the product hangs off, and it is the one a person never sees happening. The reading looks at every photograph and answers with the letters it found. Not a boundary flag per page, and not a similarity score: a list of letters, each one saying which photographs make it up, in reading order, what it appears to be, and why those photographs are one letter.

How nine photographs become six letters

The pile has no order and no grouping, and nothing about the act of photographing says which sheet belongs with which. So the pass that looks at the pictures is asked to say. It hands back a manifest: the letters it found, the photographs that make up each one, why those photographs are one letter, and which photographs are not letters at all.

WHAT GOES IN

Nine photographs, in camera order, which means nothing.

- 1 electricity bill p1
- 2 rates notice p1
- 3 electricity bill p2
- 4 a photo of her grandson
- 5 hospital appointment
- 6 rates notice p2
- 7 registration renewal (blurred)
- 8 a doctor's handwritten note
- 9 one photo holding two letters

Pages of one letter arrive apart (1 and 3), one photograph is not a letter at all (4), and one photograph carries two letters (9).

The reading

One model call. All nine photographs go in together, because the evidence for a page boundary is visual: a letterhead, the same logo as the sheet before, a different layout.

NOT IN THIS SYSTEM

- × Code that walks the photographs in sequence, sorts them, or second-guesses the division.
- × A screen that asks her to confirm this grouping. A wrong grouping shows itself on the review screen she is already reading.

THE SHAPE OF THAT ANSWER

```
{
  "letters": [
    { "photos": [1, 3], "label": "Example Energy, electricity bill",
      "reason": "same letterhead, charges table continues",
      "text": "...the whole letter, transcribed..." },
  ],
  "not_letters": [
    { "photo": 4, "reason": "a photograph of a person" }
  ]
}
```

WHAT COMES BACK: THE MANIFEST

Six letters, and one photograph that is not one. Every letter carries its photographs, its name and its reason, and the circled numbers are the photographs on the left.

- 1 3 **Example Energy, electricity bill**
same letterhead, and the charges table continues across
 - 2 6 **Calderfield City Council, rates notice**
same account number, printed page 1 of 2
 - 5 **Thornhurst Health, appointment letter**
a single sheet, complete on its own
 - 7 **Driver and Vehicle Registry, registration renewal**
letterhead legible, body too blurred to read **UNREADABLE**
 - 8 **A doctor's note**
handwritten throughout **UNREADABLE**
 - 9 **Calderfield City Council, parking infringement**
 - 9 **Kelsworth Post, collection card**
- Photograph 9 appears in both, and that is allowed on purpose. Two letters lying side by side were caught in one frame. Each letter's entry carries its own words, so how that photograph's text is split between them is the reading's judgement too.
- 4 **Not letters**
a photograph of a person, not a document **AN ANSWER, NOT A LEFTOVER**

FOUR THINGS RIDE ON THIS ANSWER

- 1 The page order is the reading's, not the camera's, because the reading can see "Page 2 of 3" printed on a sheet.
- 2 Every letter is named at birth, off the letterhead, so a queue never shows three rows all saying "reading..."
- 3 The reason is required, because nobody will ever confirm this grouping and that sentence is its only account of itself.
- 4 The words are transcribed per letter, so the photographs are looked at once and once only.

Each part of that answer earns its place. photos is which photographs, in the order the letter reads, which the reading can see because the sheet says "Page 2 of 3" and the camera never could. label is what to call this letter, taken off the letterhead, so that a letter has a name from the moment it has an id rather than three rows all saying "reading..." for six seconds. reason is required, because nobody is ever going to confirm this grouping and that sentence is its only account of itself. text is the letter's words, so the photographs are looked at once rather than twice.

not_letters is the other half of the answer. A photograph of a grandchild stays a photograph of a grandchild. No letter is invented to hold it, and one grey line on the queue says so, once, however many there were.

Why a model does this and not code. Code can divide a pile if the pile obeys a rule. This one does not: one letter's pages arrive shuffled between another's, a receipt turns up in the middle, and the same photograph can catch two letters lying side by side on a table. A rule encodes an expectation, and the defining property of this input is that there is no expectation to encode. The more chaotic the input, the more completely the answer has to be judgement, because judgement is the only instrument that works where rules have nothing to stand on.

And why the same pass that looks at the pictures. The evidence for a page boundary is visual: a letterhead, a logo, a change of layout. Almost none of it survives being flattened into text. Asking the reader for the words and then paying a second model to reconstruct the boundaries from those words is throwing away the evidence and then buying a guess.

When the manifest lands, letters are born: one row in documents per letter, one row in document_pages per photograph it turned out to own. documents carries the denormalised facts that lists and calendars need, so that drawing a screen is not three joins per row. document_pages.page_number is the reading order from the manifest, not the camera's.

No screen asks whether the dividing was right

This is the decision that surprises people, so here is the argument.

A grouping mistake does not hide. If two letters were merged, she sees one letter whose facts contradict each other. If one letter was split, she sees two letters, one of which has half its fields missing. **Both of those land on the review screen she is already looking at.** A confirmation step in front of it would catch nothing that is not caught anyway, and it would charge a person who is already overwhelmed by paperwork one more decision about a question she did not know existed. Handing the sorting back is the failure this product exists to prevent.

So the grouping is invisible. She is never told that photographs one and three became one letter. She is told there is an electricity bill for \$347.60 due on the fifteenth, and asked whether that is right.

There is one error this does not cover, and it is named rather than hidden: a real letter judged to be no letter at all. That one lands nowhere. It never becomes a document, and nothing downstream can notice what was never created. It is the sharpest part of the bet this design takes, and page 24 says how it gets measured.

STEP 3 Each letter is read

Writes: `extraction_runs` `extracted_fields`

Once the letters exist they are read one at a time and independently, so a batch of three turns into three rows that go from “reading” to “ready to check” whenever each one finishes. `extraction_runs` is one attempt at one letter: which provider, which model, what came back, and how long it took. Failed attempts are kept, because a system that deletes its failures cannot report an honest accuracy number and cannot show that a letter failed twice before it worked.

`extracted_fields` is one row per field. There are six fields, and they are the minimum a letter has to give up before it can become a task.

<code>document_type</code>	What kind of letter this is. Electricity bill, claim form, appointment letter.
<code>issuer</code>	Who sent it.
<code>action_required</code>	What she has to do, in one line, taken off the page rather than composed.
<code>due_date</code>	The day it has to be done by.
<code>amount</code>	What it costs, kept as the page wrote it.
<code>reference</code>	The number she will be asked for when she pays or calls.

Six is a floor and not a ceiling. A reader that returns more is not punished; the extra lands in an open column and is kept. But all six must come back, and a field that could not be read has to say `unreadable` rather than be left out. “I could not read this” and saying nothing are different answers, and only one of them is honest about what happened.

Two values are worth knowing about, because they look like bugs and are not. A field can come back saying **no payment is required**, or **this letter has no such thing**. An appointment letter has no reference number, and the reader still returns the field, with that constant, marked confident. It means “I looked, and it is not there”, which is a different fact from “I could not see”. The interface is free to hide such a row rather than print “Reference: not applicable” at a reader who is already working hard; hiding a row is not dropping the data.

The seventh field, and the job it came back for

The plan started with seven. The seventh was summary, one or two sentences saying what a letter is about, and it was cut before anything was built. A generated sentence is a surface for invention, and it would spend screen space on the readers least able to catch an invented sentence, at a moment when `action_required` already says what to do in one extracted line.

It came back three weeks later for a completely different job, and only because both of the original objections were about **showing it to somebody**.

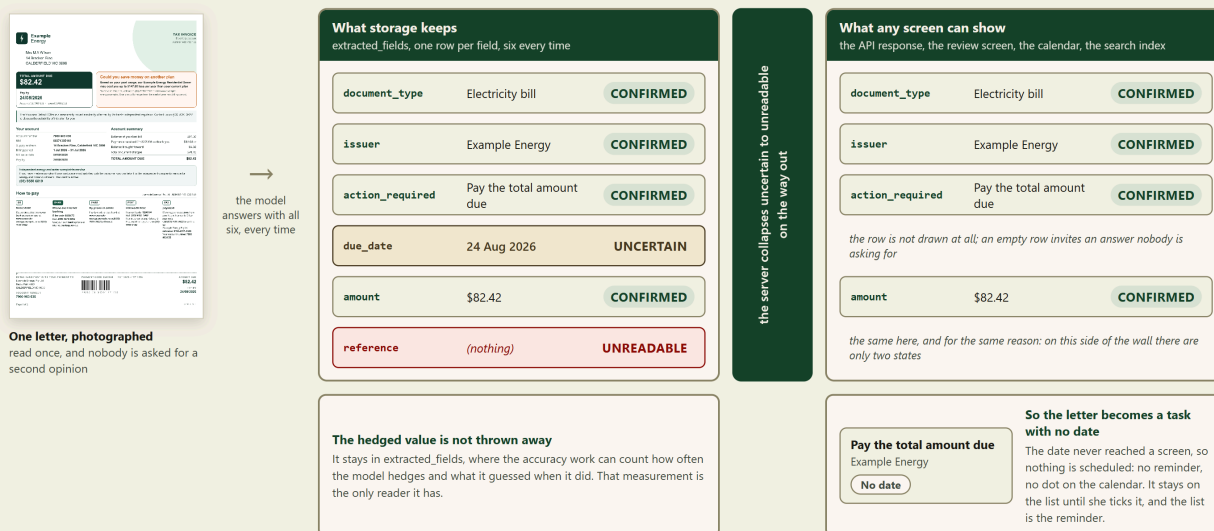
As an index it is shown to nobody. It decides what gets retrieved when the matcher goes looking for a letter, and a bad retrieval is recoverable, because the model that reads the candidates throws it out. It also turns out to be the only part of a letter written in ordinary words. Every official letter says “amount due” and “please retain this for your records”, so searching those words matches everything, which is the same as matching nothing.

So it lives on documents as a column and deliberately **not** as a seventh contract field. The contract stays at six, ExtractedFieldView never carries it, and the review screen therefore cannot show it unless somebody adds it on purpose. The same sentence that was wrong on a screen is the thing that makes the search work.

The third state, and why no screen ever shows it

Six fields, three states, and the one she never sees

One letter is read once and comes back as six fields, each carrying a status. What the database keeps and what any screen is allowed to show are not the same list, and the difference is made in one place, on the server, on the way out.



Trusting the model includes trusting its “I am not sure.” When it hedges, the product takes the hedge at face value instead of asking her to overrule it. Storage keeps the hedged value, because how often the model hedges and what it guesses is evaluation data. Nothing else keeps it: no API response, no screen, no calendar, no search index.

As far as the product behaves, a value the model was unsure of does not exist. The calendar therefore has exactly two sources, a confident reading that a person has seen, or nothing, and no procedure has to guard against a third.

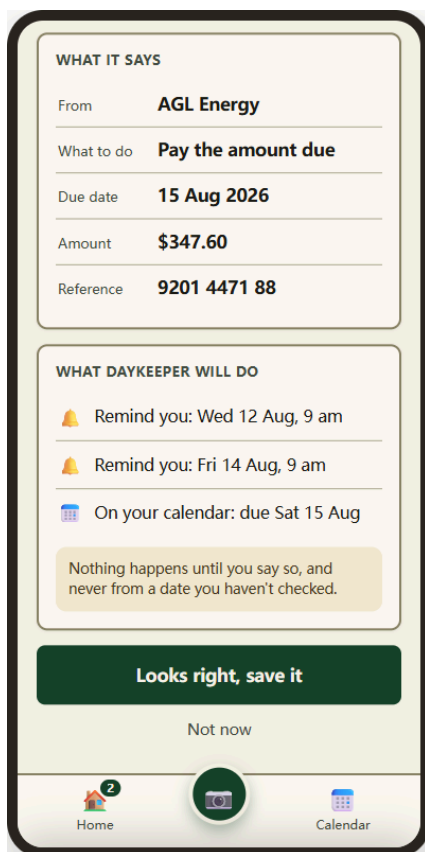
Every field carries a status: confirmed, uncertain, or unreadable. The middle one is the interesting one. It means the model produced a value it is not sure of.

Storage keeps it, because how often the model hedges and what it guesses when it hedges is exactly the data an accuracy experiment needs. **Nothing else keeps it.** On the way out of the server it collapses to unreadable. It never reaches an API response, never reaches a screen, never reaches the calendar, and never reaches the search projection. As far as the product’s behaviour is concerned, a value the model was unsure of does not exist.

Trusting the model includes trusting its “I am not sure.” The alternative is to show the hedged value and ask her whether it is right, which sounds responsible and is not. She is the person least equipped to adjudicate a model’s hesitation. The honest answers available to her are “go and find the paper letter again”, which is the thing this product exists to end, or “guess”. So the product takes the hedge at face value: it says nothing rather than something it does not stand behind.

STEP 4 She confirms

Writes: documents tasks reminders



The review screen. Every value on it was read confidently. Nothing on it is editable, and nothing on it is a question.

documents	Status becomes confirmed, with the timestamp that records a person having seen this reading.
tasks	One row, pointing back at the letter it came from, so that three weeks later “what said so?” is one tap.
reminders	One row per reminder, each carrying its own fate to record later.

The schedule

A deadline gets three reminders, seven days, three days and one day before, all at nine in the morning where she lives. An appointment gets one, the day before, because nobody needs a week of warning about a dental check-up. What separates the two is whether the letter printed a time.

Kind	Reminders	Why that
Deadline	7, 3 and 1 days before	Far enough out to act, close enough in to matter
Appointment	1 day before	The date is the event, not a window to work in
No date at all	none	The list itself is the reminder

A reminder whose morning has already gone is never planned. The bill on the screen above is due on the fifteenth and was confirmed on the tenth, so the seven-day reminder would have landed on the eighth: it is not created, and the card does not promise it. That is why the plan card and the calendar always agree with each other.

What the promise is made of

The review screen used to promise that nothing happens until she says so. It can now promise something stronger, because the promise stopped depending on anybody being careful:

The calendar has exactly two sources. A confident reading that a person has seen, or nothing.

She reads the card, it matches the letter in her hand, and she taps once. That tap turns a photograph into something on her calendar, and it writes three tables inside one transaction: the document becomes confirmed, a task is created, and the reminders are planned as rows.

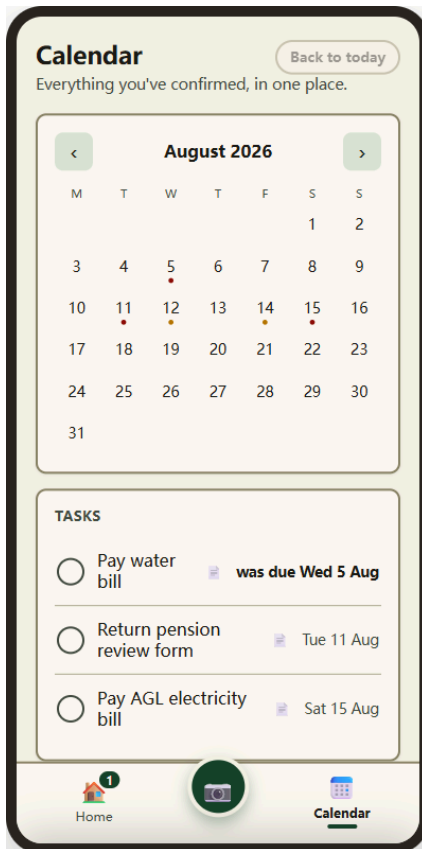
The request carries nothing at all. There is no list of fields, because nothing here can be edited. There is no list of things she acknowledged, because a value the model was unsure of never reached the screen. She looked, she nodded, and that is the entire message.

The card above the button is not decoration. It is produced by the same function that plans the reminder rows, called with the same date, so the two reminders she is promised are the two reminders that exist a second later.

An uncertain date never reaches `documents.due_date`. It never becomes a task's date, never plans a reminder, never draws a dot. There is no path by which an unchecked or hedged date can arrive anywhere, so no procedure has to guard against it arriving. This is the shape almost every safety property in this system has: not a checkpoint, a missing road.

STEP 5 The calendar

Writes: *nothing at all*



August. Red dots are due dates, gold dots are reminder mornings. The list below is the same tasks in date order.

STEP 6 She ticks it off

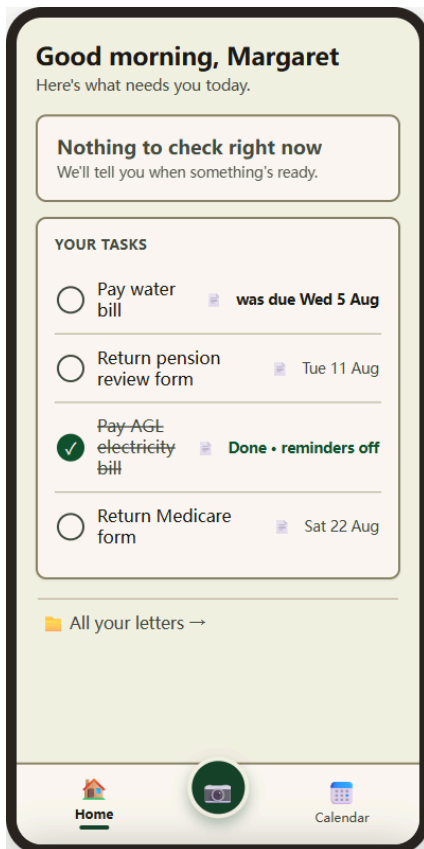
Writes: `tasks`

This step writes nothing and has no table of its own. The calendar is drawn entirely from the task list: a red dot on each task's due date, a gold dot on each reminder's morning. There is no calendar endpoint either.

That is not laziness, it is the point. A second source of dates is a second thing that can disagree with the first, on screen, in front of somebody who has no way to tell which one is lying. The calendar has no data, so it cannot disagree.

It also makes the central safety claim structural rather than procedural. Only confirmed documents produce tasks; only tasks produce dots. A letter that is still being read, or waiting to be checked, or that failed, draws nothing. Not because something checks, but because there is nothing for it to draw from.

Tapping a marked day brings up that day and nothing else.



Three faces of a task: overdue in words and in bold, upcoming plainly, completed struck through and still on the list.

She taps the circle. One column of one row changes. Nothing else in the database moves, and in particular **not one reminder row is touched**.

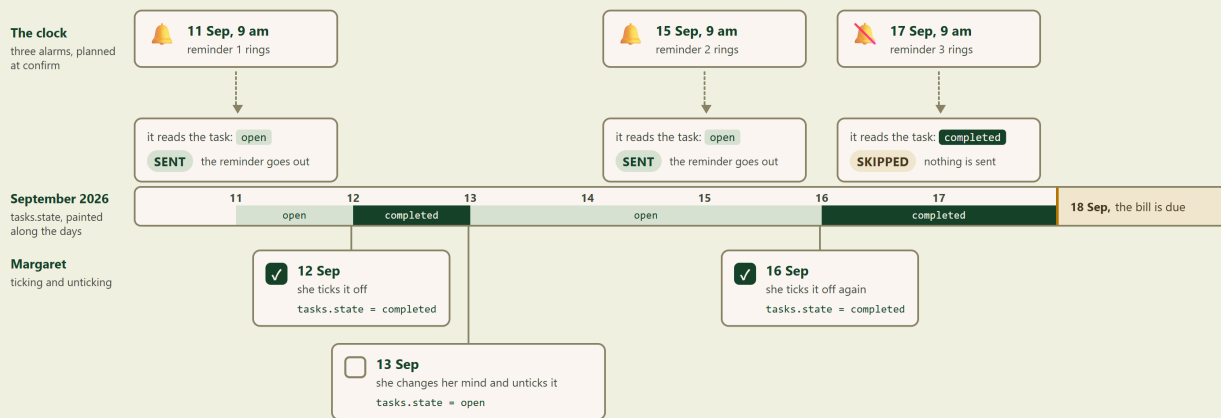
Everything else on this screen is worked out while drawing it, from that tick and today's date. Ticked means done. Not ticked with the date behind today means overdue. Not ticked otherwise means upcoming. Nothing writes the word "overdue" anywhere, so nothing has to wake up at midnight to write it, and nothing can forget to unwrite it.

Overdue is told in words, was due Wed 5 Aug, set bolder. Not in red: red already means a failed reading in this product, and colour is never the only signal for readers who may not see it. There is no badge and no separate overdue list either. Sorting by due date puts an overdue date ahead of every upcoming one, so the pinning is free.

The clock checks the tick when it rings

The clock checks the tick at the moment it rings

One bill, due 18 September, confirmed on 11 September. Ticking it off changes one column of one row and writes nothing to the reminders table. Each reminder is an alarm clock that looks at the task at the moment it rings and decides right then whether to speak. So Margaret can change her mind as often as she likes, and nothing has to be kept in step.



All three of her actions write to the same column of the same row, **tasks.state**. None of them reads the reminders table, and none of them writes to it.

reminders		written once, at confirm	
scheduled_for	status	who wrote the status	
2026-09-11 09:00	SENT	the dispatcher, at 9 am on the 11th	
2026-09-15 09:00	SENT	the dispatcher, at 9 am on the 15th	
2026-09-17 09:00	SKIPPED	the dispatcher, at 9 am on the 17th	

Three rows, planned when she confirmed the letter. Ticking wrote none of them; the dispatcher wrote the last column, each row at its own moment.

The design this replaced

REMOVED

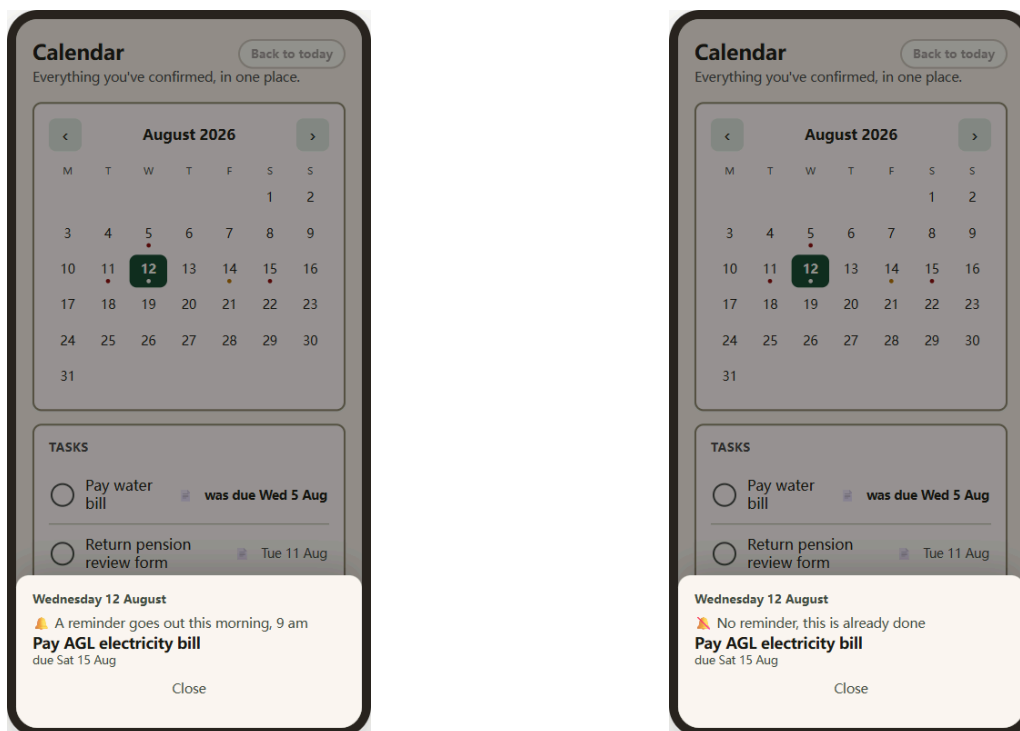
Ticking cancelled every waiting reminder, unticking revived them, except the ones whose time had already passed, which had to stay cancelled. Two writes, then two writes back, minus an exception. All of it bookkeeping, and the undo rule needed an exception within a week of being written.

A task stores one fact a person controls: whether it is ticked. Overdue, upcoming and completed are worked out while drawing the screen, from that tick and today's date in her timezone. Nothing writes the word "overdue" anywhere, so nothing has to wake up at midnight to write it, and nothing can forget to unwrite it. **Ticking is never locked:** tick at breakfast, untick at lunch, tick again at dinner, three writes to one column, no ledger and no residue.

Here is the part that is easy to build wrong. The obvious design is that ticking a task cancels its reminders, and unticking brings them back, except the ones whose time has passed, which have to stay cancelled so that nothing nags her about a deadline that is already gone.

None of that is wrong. All of it is bookkeeping, and the undo rule needed an exception within a week of being written.

So there is no bookkeeping. When a reminder's moment arrives, the clock reads the task **at that moment**. Still open means send it and write sent. Already done means send nothing and write skipped. That is the product's only judgement about whether to nag, it happens at the only moment that matters, and it lives in the one place that sends.



The same day, before and after the task was ticked. The day sheet does not store either sentence; it says what the clock would find if it rang now.

Three consequences fall out, and none of them needed to be designed:

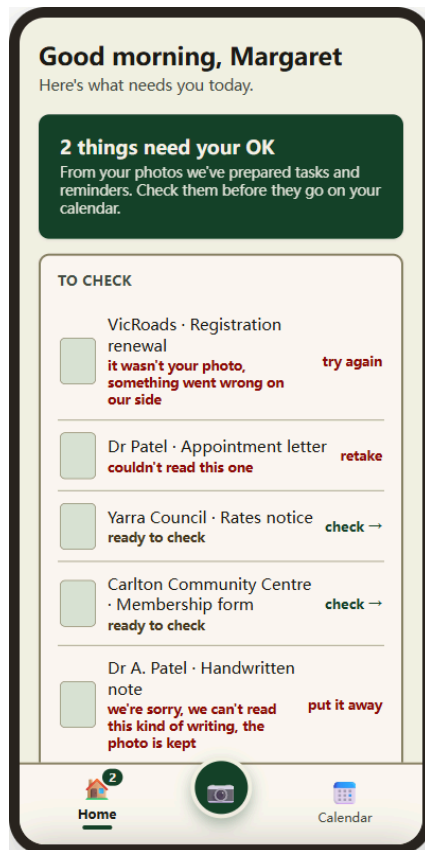
“Reminders off” is true	It is not a stored flag. There is no path by which a done task gets nagged, so the sentence is a description of the machinery rather than a promise about it.
Undo is trivial	There is nothing to undo. A reminder still ahead will find the task open when its moment comes. One already past became sent or skipped and stays so, which is not a rule, it is a description of time.
She can waver	Tick at breakfast, untick at lunch, tick again at dinner. Three writes to one column, no ledger, no residue. Ticking is never locked, on any task, in either direction, forever.

That last one was a requirement rather than a nicety. A bill due on 18 September, ticked off today and unticked tomorrow, has to just work, and so does unticking it a week after its date, which makes it overdue again by arithmetic. A product for somebody whose memory is unreliable cannot also demand that she be sure the first time.

When it does not go well

Everything so far assumed the reading worked. A product for this reader is judged on the other case, so this part is four ways things go wrong and what each one is allowed to say to her.

Three failures, three faces



Three failed letters and two good ones, in one queue. The wording differs by cause; the control differs by what she can actually do about it.

Failures are told apart by one question: what can she do about this? That is the only distinction that changes anything on screen, so it is the only distinction the product makes.

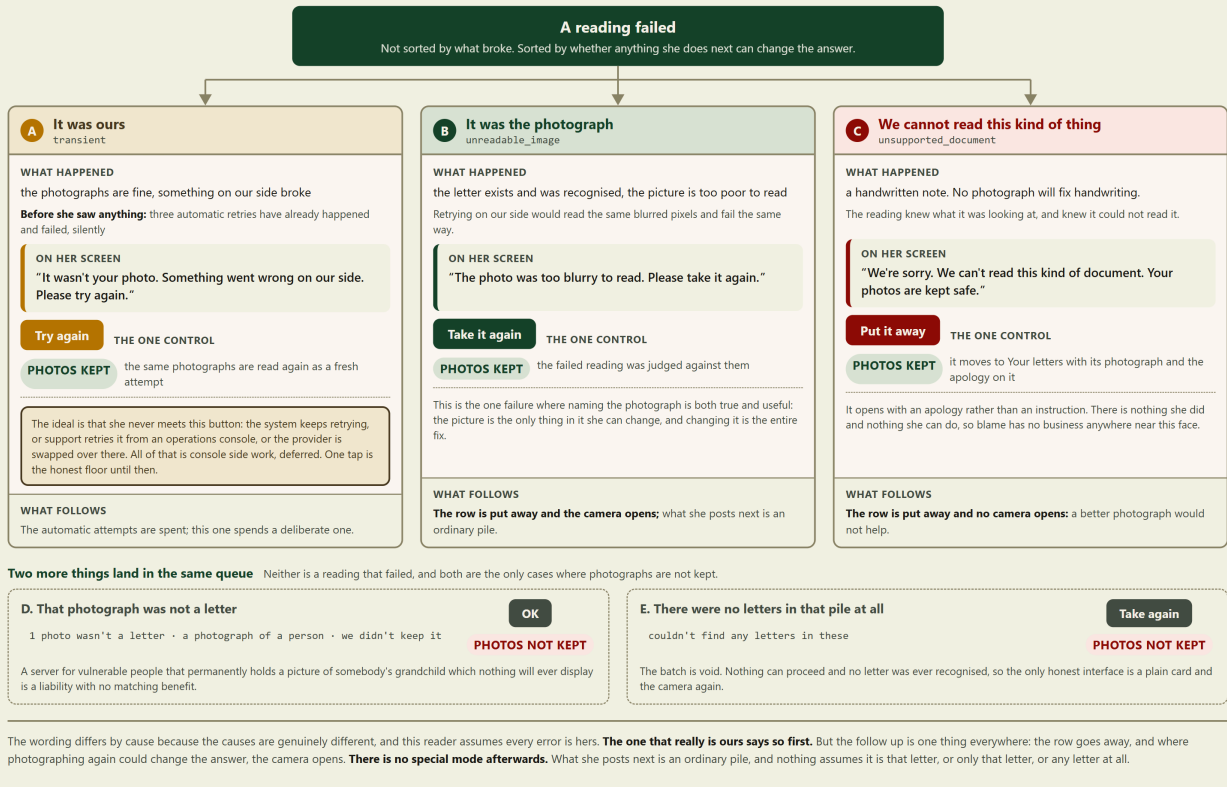
It was our fault. The photographs are fine and something on our side broke. Three automatic retries have already happened and failed, silently, before this row appears. The wording leads with “it wasn’t your photo”, because this reader assumes every error is hers, and this one really is ours. The button re-reads the same photographs; there is nothing to re-take.

The photograph. The letter exists and was recognised, the picture is just too poor to read. The pages are kept, because the reading that failed was judged against them. Her way out is to photograph it again.

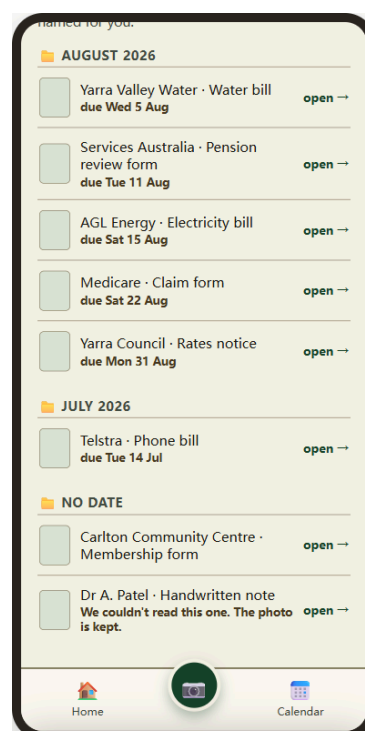
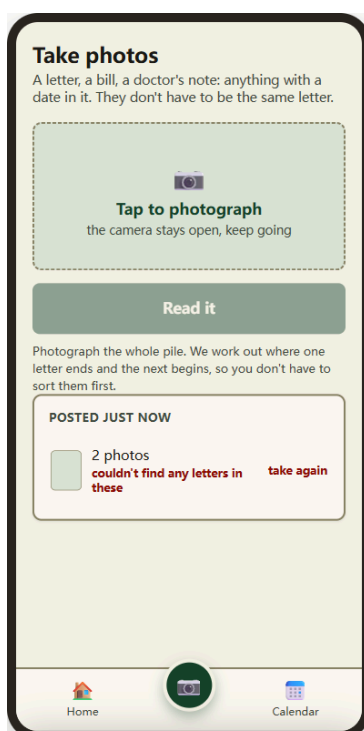
We cannot read this kind of thing. A handwritten note from a doctor. No photograph will fix handwriting. The only honest things to offer are an apology and a promise that the photograph is not lost.

Failures are told apart by one question: what can she do about it

A failed reading reaches her only after the automatic attempts are spent. What she reads then is decided by one question, and it is not a technical one: what can she do about this? Three answers, three sentences that share no words, and one identical follow up.



The button she should never meet. "Try again" is a compromise and the record should say toward what. The ideal is that a transient failure is invisible: the system keeps retrying in the background, or somebody retries it from an operations console, or the provider is swapped over there when one of them is down. None of that is something she can be asked to know about. All of it is console-side work, deliberately out of scope this semester, and the one tap is the honest floor until it exists.



Left: a pile with no letters in it at all. Nothing can proceed, so a plain card says so and the camera is the only button. Right: where the apology points. The letter we could not read is in her letters, with its photograph, and the note says exactly that.

Two of these throw the photographs away, and the reasoning is the same in both cases. A photograph the system has decided is not a letter, or a batch with no letters in it, will never be shown on any screen again. A server for vulnerable people that permanently holds a picture of somebody's grandchild which nothing will ever display is a liability with no matching benefit.

After any of these, there is no special mode. The row goes away, and where photographing again could change the answer, the camera opens. What she posts next is an ordinary pile: nothing assumes it is that letter, or only that letter, or any letter at all. If the re-photographed letter should be reconnected to what is already here, recognising it is the reading's judgement, not a wire we soldered in advance.

The letter that names no date

Check what we found
Carlton Community Centre · Membership form

WHAT IT SAYS

From **Carlton Community Centre**

What to do **Return the signed form**

Reference **Member 2214**

WHAT DAYKEEPER WILL DO

No date on this letter. It's saved; nothing goes on your calendar.

Nothing happens until you say so, and never from a date you haven't checked.

Looks right, save it

Not now

Home Calendar

Sometimes the letter simply does not say when. Sometimes it does and the photograph would not give it up. From the product's side these are the same situation, and it has one answer for both.

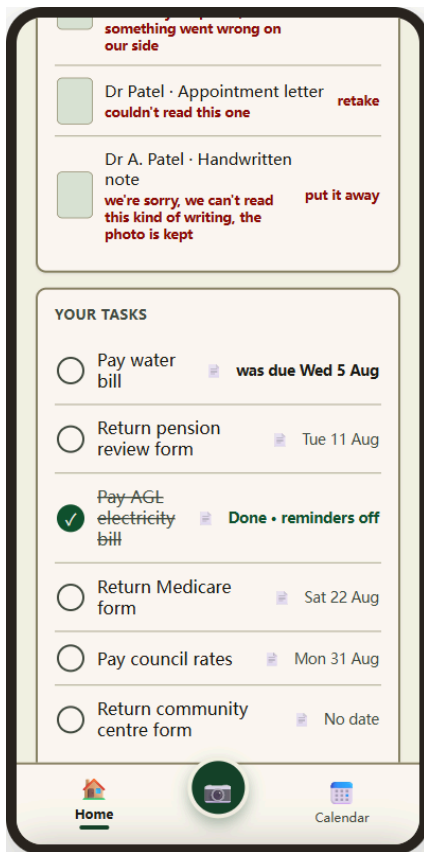
The letter still becomes a task. It sits in her list saying No date, never reminds her, never touches the calendar, and stays there until it is ticked. The list itself is the reminder.

Notice what the screen does not do. There is no empty box inviting her to type the date, no amber warning, and no question mark. An empty row invites an answer that nobody is asking for, so the row is not drawn; the card states the situation once, in a sentence, and moves on.

A letter with no clear action at all does not become a task. It is kept as a letter, with its photograph, and that is the whole of it.

A letter with a clear action and no date anywhere on it. The row for a date is not drawn at all, and the plan card is a sentence with a full stop.

Why she cannot type the date in herself. It is the obvious feature and it was deliberately cut. For this reader, recognising a value on a card and typing a date into a box on a phone are not the same size of task, and the box would be demanded at the worst moment, about the values least likely to be right. So the product keeps two verbs. Whatever is wrong or missing, her one move is photographing the letter again, through the same camera as everything else, and the matching step is what connects the new reading to the old letter. The feature is deferred rather than rejected: if it comes back it comes back as a designed thing, not as a text box that happened to be lying around.



The full list. Overdue at the top because its date is smallest, the dateless one at the bottom because it has no date to sort by.

Pages that arrive later

Writes: `matching_runs`

She uploads three pages of a form, and five minutes later finds the fourth in a drawer. She photographs it. Nothing about that photograph says it belongs with the other three.

So the question the system has to answer is: are these pages the rest of a letter that is already here? The obvious implementation is a query. Find documents from the same sender, or with the same reference number, in the last few months. It works on an electricity bill and falls over on the first thing that is not a bill.

Letters are not a genre. A party invitation has a date, a place and a thing to do, and it has no issuer worth matching on and no reference number at all. The save-the-date it belongs with says “Carlton” where the invitation says “45 Elgin Street”: not one word in common. A query we write in advance has to anticipate the shape of somebody’s correspondence, and there is no shape.

The model searches, code does not narrow first

The matcher gets tools and a question rather than a result set. Three tools, chosen because they are the three any model is most fluent in:

<code>glob(pattern)</code>	Find candidate letters by path
<code>grep(pattern, ...)</code>	Look inside them, with the path on every hit
<code>read(path)</code>	Read one in full and decide

The dateless task is not a special case in the sort, it is the natural end of it. Due date ascending, and a task with no date goes last. One rule gives the overdue pinning and the dateless placement together, and neither of them is a line of code that could rot.

This screen is also where the correction story ends up. If the model reads an amount confidently and gets it wrong, and she notices, she photographs the letter again. The new reading arrives, the comparison on the next page sees that the amount changed, and the letter comes back to be looked at. Correction is not a separate feature; it is the ordinary flow, run again.

What that costs is worth naming. User corrections were going to be the in-product accuracy signal, and with editing gone that signal is gone. The numbers now come entirely from the synthetic evaluation line, where the truth is known by construction.

What they search is a **view** over the database, not a copy of it. That distinction is the whole design. A copy written alongside the source drifts, and a drifted index does not fail loudly: it answers “nothing matches”, confidently, about a world that has moved on. A view cannot drift, because it is not a copy, it is the query.

The view presents each letter as a small document with a path built from its due date and sender, front matter carrying the six fields, and a live line saying what became of it (open, due 2026-08-15, completed 2026-08-02, or none). That last line is derived at read time rather than stored, for exactly the same reason as everything else in this document: a stored flag would need updating on every tick and untick, and a stale flag does not error, it confidently lies.

Four values decide whether she is interrupted

When the late pages are placed against a letter she already confirmed, the letter is read again as a whole, and then the new reading is compared to the stored one on four values.

action_required	What she has to do
due_date	Which day
due_time	What time, if it is an appointment
amount	How much

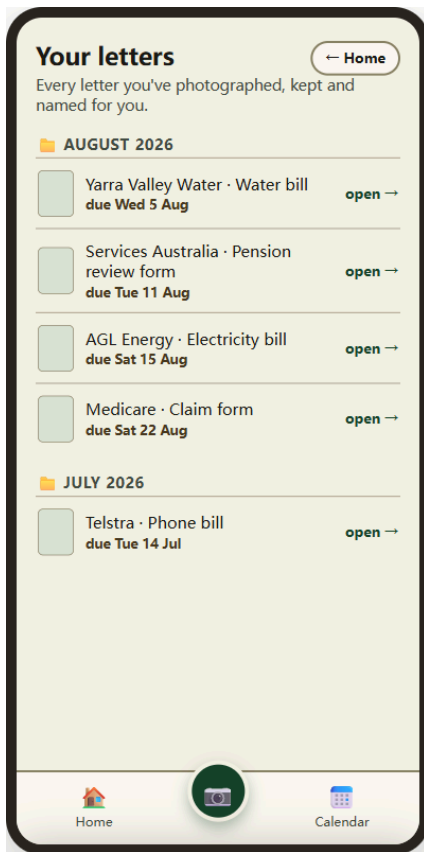
If any of them changed, the letter goes back to needing review and she is asked again, in a row that looks exactly like a new letter. If none of them changed, the letter is updated silently and she is not told anything.

That comparison is code, not judgement. Whether these pages belong to that letter, and what the letter now says, are the model’s answers. Whether that changes what she has to do is != on four values. Keeping it that way means the rule can be demonstrated and tested, instead of being a second probabilistic surface with no ground truth behind it.

amount is in the list even though it is not literally “what and when”. She still has to pay the AGL bill on the fifteenth whether it is \$347 or \$412, but paying the wrong number is a harm she carries.

Two things follow that are easy to miss. First, the letter’s identity is never on screen: she is never told “this was a late page for the letter you checked on Tuesday”, because that is our housekeeping and it costs her attention to read. Second, every silent update is recorded in `matching_runs`, because it is the one class of decision this system makes on her behalf that is invisible by construction. If it is ever wrong, nothing else will surface it.

Your letters



Months as folders, newest first, each letter named by what the reading found. Reached from a quiet line at the bottom of Home, not a fourth tab.

Two screens in this product promise that a photograph is kept. This is where that promise is kept.

It is the search projection, made visible. The months are the first level of the path the matcher globs over; the names are the sender and the type it read off the letterhead. Nothing has been renamed in storage, and there is no filing system underneath: the folder view is a rendering of the same letters, computed from the same reading.

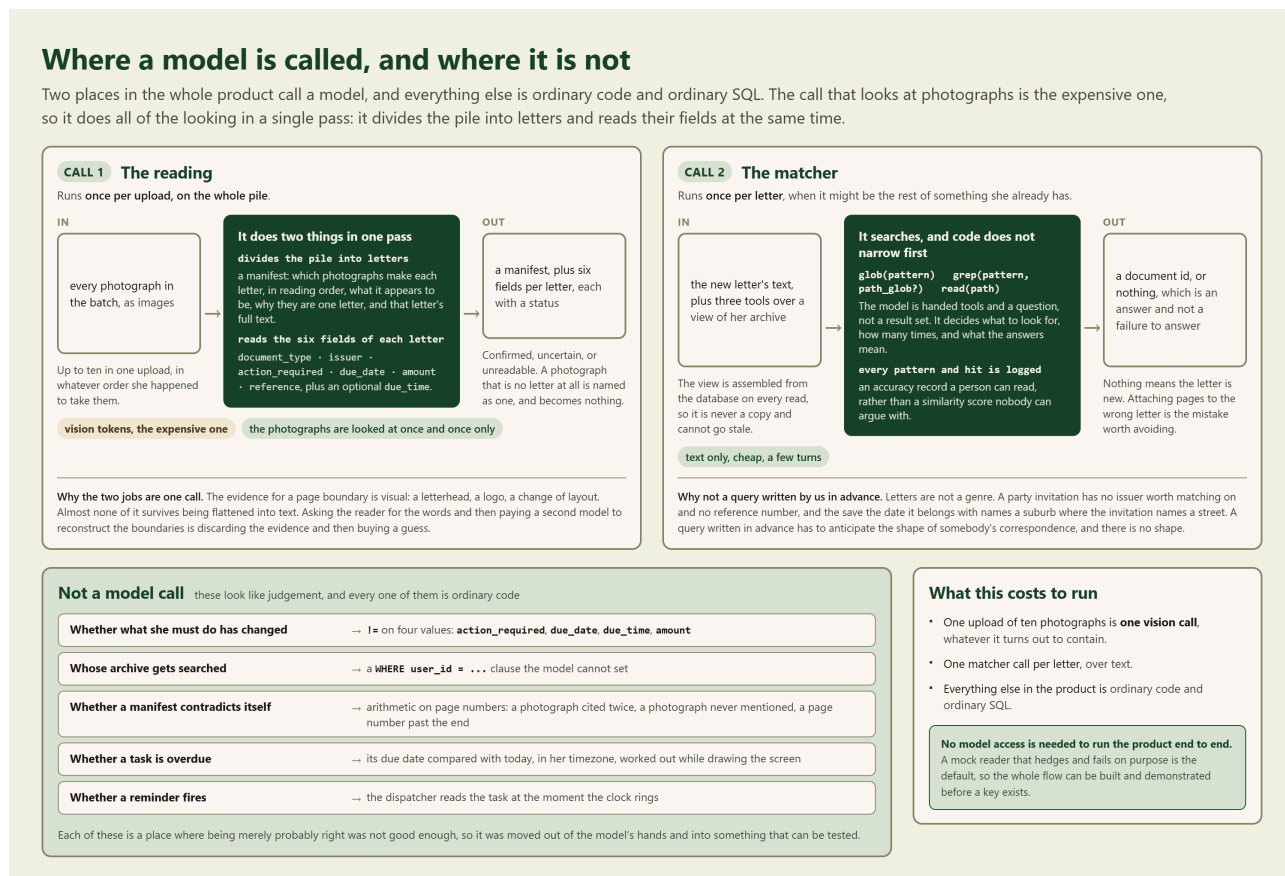
Letters that never became a task live here too. The one we apologised for is in the No date folder with its photograph and its note. So is the membership form with no date on it, next to the task it made.

It is deliberately a quiet door. She can ignore this screen for a year and lose nothing, which is why it is a line at the bottom of Home rather than a tab competing for attention with the two things she actually has to do.

Where a model is called, and where it is not

The walk is over, so it is worth saying plainly where the intelligence in this system actually sits, because “an AI app” usually means something vaguer than what is going on here.

There are exactly two places a model is called. One looks at the photographs. One searches what she already has. Everything else in the product is ordinary code and ordinary SQL.



Two things on that picture are worth pausing on.

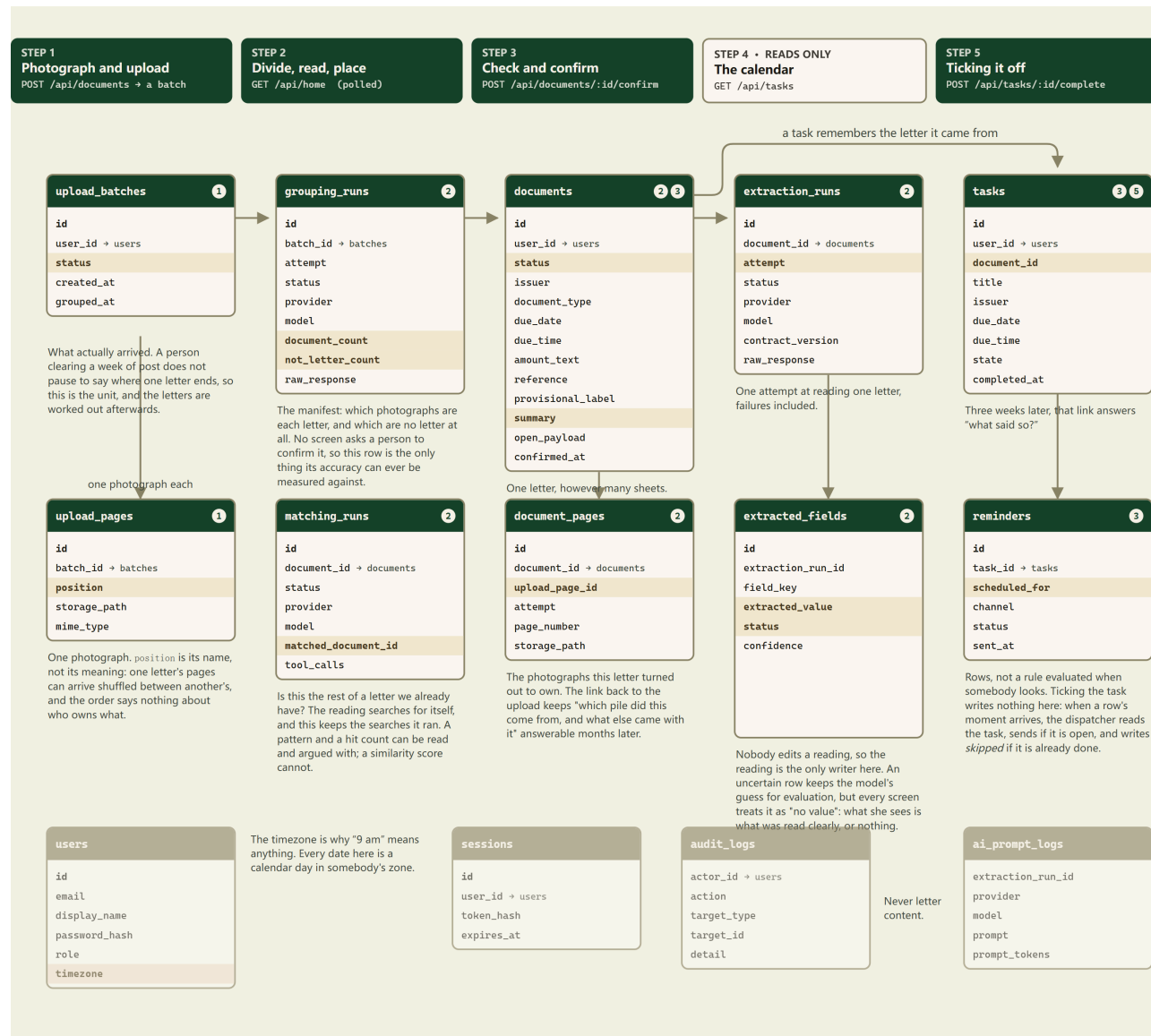
The photographs are looked at once. The pass that divides the pile is the same pass that reads the fields, because the evidence for a page boundary is visual and almost none of it survives being flattened into text. Splitting that into two calls would mean paying twice and paying the second one to reconstruct what the first one could already see.

The seams are code, on purpose. Whether these pages belong to that letter is judgement, and it is the model's. Whether what she has to do has changed is != on four values. Whose archive gets searched is a WHERE clause the model cannot set. Whether a manifest contradicts itself is arithmetic on page numbers. Each of those is a place where being probably right was not good enough, so it was moved somewhere it can be tested.

One consequence is worth stating for anyone deciding how much model access this project needs: **the product runs end to end without any**. The default reader is a mock that takes a few seconds, hedges about the due date roughly half the time, cannot read the reference about one time in five, and fails outright about one document in eight. That is deliberate. A mock that always succeeds instantly produces an interface with no waiting state, no failure path and no hedging to handle, and all three of those are where this product actually lives. Model access changes the accuracy numbers, not whether the thing works.

The fourteen tables, and when each one is written

The five steps along the top are the walk you have just taken. The ten tables under them carry a badge saying which steps write them; the four at the bottom sit off the path a letter travels. Step four has no badge anywhere, because the calendar writes nothing at all.



The decisions behind all of this

Everything in this document came from one of eight written decisions. They live in docs/architecture/ in the repository, one file each, and each one records what was decided, what it was decided against, and what would have to be true for it to be wrong. When something in the code looks odd, the reason is in one of these rather than lost in a chat log.

ADR	What it settles	Where you saw it
001	One Next.js application. The pages and the API are one project, deployed as one thing. No second service to keep in step.	p. 4
002	Our own users table and sessions as rows, reached through exactly one function, so that adopting a managed provider later replaces one file.	p. 5
003	Plain PostgreSQL, one schema file, no migrations, and the photographs in an S3 bucket from the first day rather than on a disk we would have to unwrite.	p. 5
004	Six fields as the floor, one provider interface, and a mock reader that fails and hedges on purpose so the interface has to handle both.	p. 8
005	An upload is a pile, and the reading decides where the letters divide. No screen asks a person to confirm that division.	p. 6
006	The matcher searches its own archive with glob, grep and read, over a view of the database rather than a copy.	p. 18
007	The tick is the only state a task stores, and the clock checks it at the moment it rings.	p. 11
008	The product shows, it never asks. Nothing is editable, hedged values never reach a screen, and the camera is the one remedy.	p. 9

Two habits run through all eight, and they are worth stating on their own because they explain decisions this document has not had room for.

Every guard is becoming a missing path rather than a checkpoint

The calendar cannot show an unchecked date, because the calendar has no data of its own. The dispatcher cannot nag about a finished task, because it reads the task at the moment it rings. A hedged value cannot be acted on, because it is never allowed to become actionable in the first place. None of those are enforced by a check that somebody has to remember to write, and none of them can be broken by somebody forgetting.

This is a deliberate direction rather than a coincidence. A checkpoint is a promise that everybody will keep calling it. A missing path is a promise the architecture keeps whether anyone is paying attention or not, and on a five-person team over eight weeks that difference decides which promises are still true at the end.

The more chaotic the input, the more the answer has to be judgement

A person photographing a week of post produces input that obeys no rule, and every attempt to write one down smuggles in an assumption nobody agreed to. That is why the dividing is the model's answer and not a loop over page boundaries, and why finding a letter in the archive is a search the model runs rather than a query we wrote in advance.

The other half of the habit matters as much: **where judgement ends, code begins, and the seam is always something binary**. Whether these pages belong to that letter is judgement. Whether what she must do has changed is != on four values. Whether a manifest is self-consistent is arithmetic on page numbers. Whose archive gets searched is a WHERE clause the model cannot set. Each of those is a place where being merely probably right was not good enough, so it was moved out of the model's hands and into something that can be tested.

What is not settled

These are open questions rather than oversights. Each one needs a decision nobody has made, and guessing at it now would mean building the wrong thing twice. They are grouped by how soon a person meets them.

The four a person meets first

One row becoming several	A pile is one row while it is being divided and then it is three letters. Whether the row splits in place, fades and is replaced, or the letters slide in underneath, is a piece of motion nobody has designed, and it is the first thing she sees after pressing the button.
Holding the camera open	The capture screen says the camera stays open and shots accumulate. A plain file input cannot do that: on both iOS and Android it closes the camera and returns to the page after every single shot. A live stream captured to blobs can. Whichever is chosen has to be said on the screen as well as in the code.
Which page was the bad one	A five-page letter that fails says only that the photograph was too blurry. Which one? A page number on the failure would let it say “page three”, and she would photograph one sheet instead of five.
The limits, seen from the camera	Ten photographs, ten megabytes each, and a current phone clears that per frame routinely. Whether the client scales a photograph down to fit or refuses it, and what the screen says at the tenth, is undecided. Refusing quietly loses a photograph she deliberately took, which is the failure the limits exist to prevent.

The machinery behind the screens

What runs a reading	The upload answers before the reading happens, so something has to perform it: in the same process after responding, a sweep over an index of queued runs, or a real queue. The schema supports all three.
What wakes the clock	What the dispatcher decides is settled: at a reminder’s moment it reads the task, sends if it is open, writes skipped if it is done. What wakes it up is not. A cron job, a platform scheduler, or in-app only.
The “your letters are ready” message	The prototype promises one batched message when readings finish. It is not modelled yet: reminders belong to tasks, and these documents have no task. It needs a table of its own, which arrives with the decision about batching window, channel and quiet hours.
A blur check at capture time	Catching a bad photograph while she is still holding the letter would be worth a great deal. It needs a second provider method and a model that can actually do it, and it arrives with that model or not at all.

Around the edges

Password reset needs an email sender. The admin endpoints need the operator’s permissions settled first, and the promise on screen is that letter content is never visible there, which the audit table is built to make impossible rather than merely forbidden. A settings screen would expose the timezone the schema already stores. Sign-in has no rate limiting, which matters before anything is public. And the greeting says “Good morning, Margaret” using a display name that holds “Margaret Whitfield”, so a preferred name arrives whenever somebody decides what this product calls her.

Two numbers this design is staked on

Everything else on this page is a decision waiting to be made. These two are measurements waiting to be taken, and they are different in kind: the design above is **betting** on them, and if either comes back badly the right response is to revisit a decision rather than defend it.

How often the dividing is wrong	No screen confirms the grouping, so this number is the whole safety case for step 2. It has three parts and they are not equally dangerous. A merge shows itself as one letter whose fields contradict each other, a split as two letters with half their fields missing, and both land on the review screen she is already reading. A real letter judged to be no letter lands nowhere , and nothing downstream can notice what was never created. That is the part that matters most, precisely because it is the only one with no second pair of eyes anywhere behind it.
How often the matcher fails to find a letter that was there	This decides whether the search stays literal or has to climb: trigram matching first, embeddings after that. Both rungs are designed and neither is built, because the number that would justify one does not exist yet.

Neither measurement needs a real letter or a real person. Batches of known composition can be assembled from synthetic documents and the reading's answer compared against the truth, which is a binary judgement per photograph. The same holds for the matcher: hide a page from a letter that is already in the archive, hand it back as a new upload, and see whether it comes home. The ground truth is known by construction, which is what makes these numbers reportable rather than anecdotal.

Where to start reading

Every one of these is a real file in the repository. The point of this table is that you should not have to read the other eight to change one thing.

If you want to	Read this first
Run the whole thing on your machine	<code>docs/start-here.md</code> : what to install, what the seed contains, and where everything lives
See what a screen is supposed to look like	<code>docs/prototype/user/daykeeper-sketch-live.html</code> , which is the UX authority. Where it and a document disagree, it wins
Build an endpoint	<code>docs/api.md</code> for the shape, <code>src/lib/contract/</code> for the types. Import the types rather than restating them
Change the database	<code>db/schema.sql</code> , which is the only definition there is, then <code>npm run db:reset</code>
Style anything	<code>docs/theme.md</code> before touching a colour. It has the palette, the measured contrast of every pair, and why each rule exists
Understand why something is odd	<code>docs/architecture/</code> , the eight decisions listed on page 23
Connect a real model	<code>src/server/extraction/</code> , behind one provider interface, with the mock as the worked example
Touch a photograph	<code>src/server/storage.ts</code> , which is the only place in the application allowed to read or write bytes

The repository holds the schema, the contract and its validators, a mock reader that misbehaves on purpose, the prototype, the theme with tests that fail when a colour drifts, and the eight decisions. The endpoints in `docs/api.md` describe the shape they are to be built in. That specification exists first on purpose: the interface, the reader and the database are being written by different people, and the only way that ends in something that fits together is to settle the seams before the parts.